CLIMB at VBS 2027: Multi-Signal Video Retrieval with Team-Based Collaboration

Tobias Wurzer[1(✉)][0009−0002−0965−6877], Sebastian Eisner[1][0009−0003−0855−9116], and Klaus Schoeffmann[1][0000−0002−9218−1704]

Klagenfurt University, Klagenfurt, Austria
tobias.wurzer@aau.at,sebastian.eisner@aau.at,klaus.schoeffmann@aau.at

Abstract. We present CLIMB, an interactive video retrieval system built for the Video Browser Showdown 2027. CLIMB retrieves information across four complementary signals: visual embeddings, on-screen text, generated captions, and speech transcripts. By combining rank-based fusion with a hybrid of lexical and semantic matching, the system ensures broad retrieval coverage. Additionally, queries can be chained into temporal sequences, providing increased flexibility while sharply narrowing the result set. We treat retrieval as a shared team activity, allowing operators to send hints to each other and automatically hiding teammate-submitted scenes for ad-hoc video search. Crucially, core functions such as search, browsing, and submission remain fully operational even if the shared collaboration service becomes unreachable.

Keywords: Video Browser Showdown · Interactive Video Retrieval · Collaborative Search · Semantic Embedding.

1 Introduction

The Video Browser Showdown (VBS) [5, 6, 11] is an interactive video retrieval competition that asks participants to solve known-item search (KIS), ad-hoc video search (AVS), and visual question-answering (VQA/QAS) tasks against large-scale video collections. For VBS 2027, the main collection is V3C [12] with 28,450 videos, roughly 3,800 hours, and about 8.7 TB of full-resolution video data, along with the smaller collections MVK [17] and GynSurgLHE (LHE) [8].

Successfully completing these tasks presents multiple fundamental challenges. A single visual embedding is often too vague to capture precise task hints, such as specific on-screen text or spoken dialog, and for difficult tasks, teammates have no direct way to point each other to promising results. Additionally, because AVS scoring rewards distinct correct shots, a team of n operators searching independently spends a substantial share of its submissions re-finding what a teammate has already sent.

CLIMB addresses all of them. It indexes every video as scenes and keyframes, retrieves over four independent signals, fuses them by rank, allows for operators to send clues to their teammates, and treats an AVS task as a single shared activity, vanishing duplicate scenes from everybody's list.

Concretely, we contribute:

1. Keyframe-level retrieval fusing four heterogeneous signals by rank (Sect. 4.1).
2. Temporal chaining, `A >> B >>(120) C`, with a full query per stage (Sect. 4.3).
3. Network-resilient team collaboration: shared AVS state and hints (Sect. 6).
4. An index engineered for commodity hardware: binary-quantized HNSW with oversample-and-rerank (Sect. 3).

2 System Overview

CLIMB is built from six components (see Fig. 1). Decoding each video, segmenting it into scenes, selecting keyframes, computing embeddings, and text recognition in frames via OCR, as well as captioning and transcribing, happens in an offline pipeline described in Section 3. The resulting scenes, keyframes, and four indexes that the retrievers query land in a PostgreSQL database.

At query time, the system runs as two processes, a Node.js backend transmitting media to the frontend, handling DRES [10, 14] communication and collaboration features (Sect. 6), as well as a Python engine handling models, query embedding, and rank fusion. Both connect to the database: the backend reads metadata directly, delegating only search execution to Python. The frontend (Sect. 5) communicates exclusively with the Node.js backend.

For scenarios where network reliability is a concern, CLIMB supports fully localized deployments, serving media assets directly from localhost. Additionally, the shared and optional collaboration service (Sect. 6) only transmits a few kilobytes of JSON polled every four seconds.

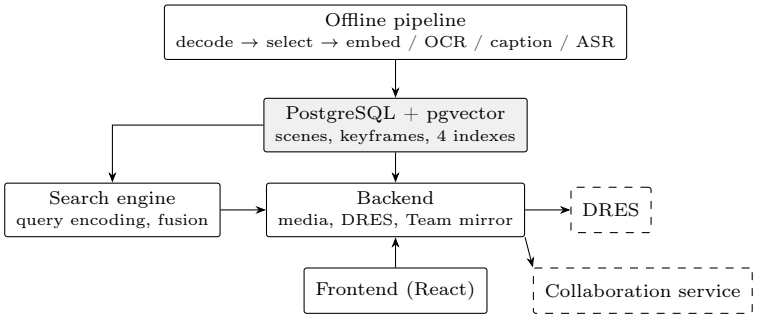


Fig. 1. CLIMB architecture. All components can run locally on each machine, except for two external services: DRES and the collaboration service, which is deployed once.

3 Indexing Pipeline

3.1 Scenes and Keyframe Selection

The first step of the indexing pipeline is shot segmentation and keyframe extraction. For all three datasets, scene boundaries are shipped with the dataset.

The segmentation has been done with TransNetv2 [16] for V3C and a specific keyframe extraction algorithm [15], based on ORB keypoint tracking [13] for the LHE [8] and MVK videos [17]. MVK clips from the same dive are shipped as separate clips, but they are grouped into one recording on a shared timeline, so exclusion, temporal chains and AVS coverage work per dive, while DRES submissions still reference the individual clips.

Within each scene, candidate frames are decoded at 2 fps, and the scene receives one keyframe per four seconds of duration, at least two and at most 32. Fades and blank frames (very dark, very bright, or low-contrast) are discarded first, unless they contain edges, so that title cards and credits remain available for OCR. Each remaining candidate is described by a cheap, model-free fingerprint: a colour histogram, a perceptual hash, and the edge density. Selection follows a greedy farthest-point strategy [3], seeded with the frame closest to the scene's mean fingerprint.

3.2 Feature Extraction

CLIMB extracts four signals, two per keyframe (visual, OCR) and two per scene (caption, speech). OCR uses PP-OCR [21], while the 1024-dimensional visual embeddings are extracted with SigLIP2 [18]. For LHE, we also extract embeddings using PeskaVLP [20], as it was trained on surgical imagery. We also tried using BioCLIP 2.5 [4], trained on biological imagery, in order to improve retrieval on the MVK dataset, but it did not yield any improvement (Sect. 7).

Given the high computational cost and intra-scene visual redundancy, captioning is restricted to each scene's first keyframe. For full-scale captioning, we apply Qwen3-VL-4B-Instruct [1]. Audio transcription is handled by Whisper large-v3 [9]. Transcriptions produced from music or noise, identified as repetitive or isolated from other speech, are excluded from retrieval. In order to encode text into semantic embeddings, we use multilingual-e5-large [19], which performs encoding with 1024 dimensions. For efficient retrieval, all generated embeddings are stored in an HNSW index [7] over binary-quantized vectors. To maintain search accuracy while minimizing memory overhead, initial search candidates are reranked against their full-precision vectors. This quantization significantly reduces the memory footprint, requiring only 5.3 GB for 12.4M keyframes compared to 28 GB for half-precision storage.

4 Retrieval Model

4.1 Rank-Based Fusion at Keyframe Level

As discussed above, we use four different encoders to encode as broad a spectrum of information as possible. In order to avoid sensitive and model-specific calibration, we use a rank-based fusion approach, explained below.

Each signal returns a list of ranked objects. While the visual and OCR signal natively return a list of ranked keyframes, the caption and audio signals are

scene-based. Thus, each keyframe inherits the rank of its parent scene. The keyframes are then merged via weighted Reciprocal Rank Fusion (RRF) [2]. This works similarly to normal RRF but weighs the signals depending on their source, allowing us to highlight keyframes found via specific encoders, i.e., OCR, where a lexical hit is much more likely to belong to the correct scene than a similar visual embedding. As by [2], the fused ranking score for a keyframe $d \in D$ is defined as:

$$RRFscore(d \in D) = \sum_{r \in R} \frac{w(r)}{60 + r(d)} \tag{1}$$

Where $r(d)$ is the rank assigned by encoder $r \in R$ with weight $w(r)$. R denotes the set of information modalities. The constant 60 reduces the dominance of top-ranked keyframes within individual information modalities, allowing evidence from multiple sources to contribute to the final score. This constant was empirically determined in [2]. For V3C, the uploader's video description and tags are additionally matched against the query as a video-level prior, adding an additional RRF term for each already retrieved keyframe's video's rank, re-ordering the results. The system's default weights are 1 for visual, caption, speech and metadata, 4 for OCR and 8 for explicit matches via the `said:` or `text:` operator (Sect. 4.3).

4.2 Semantic and Lexical matching

Although cross-modal visual retrieval with SigLIP2 is inherently semantic, the remaining text-to-text signals could theoretically rely on lexical matching. However, strict keyword matching on generated captions suffers from severe vocabulary mismatch. For instance, a query for "law enforcement" shares zero lexical overlap with a highly relevant caption describing "a police uniform". In fact, the terms "law" and "enforcement" never appear within our extracted captions.

While ASR transcripts are embedded semantically for conceptual and cross-lingual search, users can invoke a *said:* operator for exact lexical matching of specific quotes. In contrast, OCR operates as a purely lexical signal to pinpoint exact on-screen text provided in task hints. Semantic matching here is redundant, as the visual embedding already captures general concepts.

4.3 Query Language and Temporal Chains

A plain text query searches all four signals at once, and the operator prefixes in Table 1 narrow the search down. Each adds a requirement at video level, and the plain text is then ranked only within the videos that satisfy all of them. Additionally, once the prefixes leave at most 300 videos, the embedding-based signals search these exhaustively instead of using the index.

Some tasks describe not a single scene but an ordered sequence, such as a bridge followed by a beach. For these, `>>` joins queries into a temporal chain. A query `A >> B >> C` has three stages, each a complete query including its own prefixes, so `text: pegasus >> mountain bike race` finds a scene showing the text

"pegasus", followed by one that matches a mountain bike race. By default, consecutive stages must lie in the same video and at most 60 seconds apart. An annotation such as `(120)` widens the gap to 120 seconds, up to a maximum of ten minutes, with at most four stages per chain.

Each stage returns a ranked list of keyframes, grouped by video. Videos lacking a keyframe for any stage are excluded. For every first-stage hit, the best continuation is found by working backwards from the last stage, so each pair of adjacent stages is compared only once. At most three chains are kept per video, and all chains are ranked by their score, the sum of their stages' RRF scores.

Table 1. Query syntax. Prefixes are video-level requirements. They are repeatable and can be combined. `>>` chains queries temporally within a single video.

Input	Effect
`a drawing of people on a beach`	Global search (all signals)
`text: state capitol`	Scenes showing this text
`said: new york`	Scenes in which this is said
`tag: travel`	Only videos tagged with `travel`
`video:00191 video:00192`	Only these videos
`-video:00191`	Exclude this video
`a video game sunset >> door`	Sequential search (same video)

5 Interactive Search Interface

The CLIMB user interface is shown in Figure 2. A search typically follows an iterative interaction loop. First, the operator selects the target collection (V3C, MVK, or LHE), which restricts all results to it. The operator then enters a query that returns a grid of keyframes. Selecting one displays a five-second scene preview starting at the corresponding timestamp. The filmstrip located under the video player provides an overview of all the keyframes, allowing efficient navigation between them. Keyframe times are automatically populated but remain editable, to fine-tune the target submission frame. In addition, there are a variety of buttons for quick search refinement. To keep this cycle responsive, query results are cached and paginated: the initial query executes the search, while subsequent pages are loaded from cache upon scrolling. *Submit to DRES* sends the selected item to the evaluation server. *Exclude video from search* removes a video from the results, and when textual queries fall short, *Find Similar* enables visual query-by-example. Finally, for VQA tasks, operators type the answer below the filmstrip and submit it as text alone or together with the selected shot.

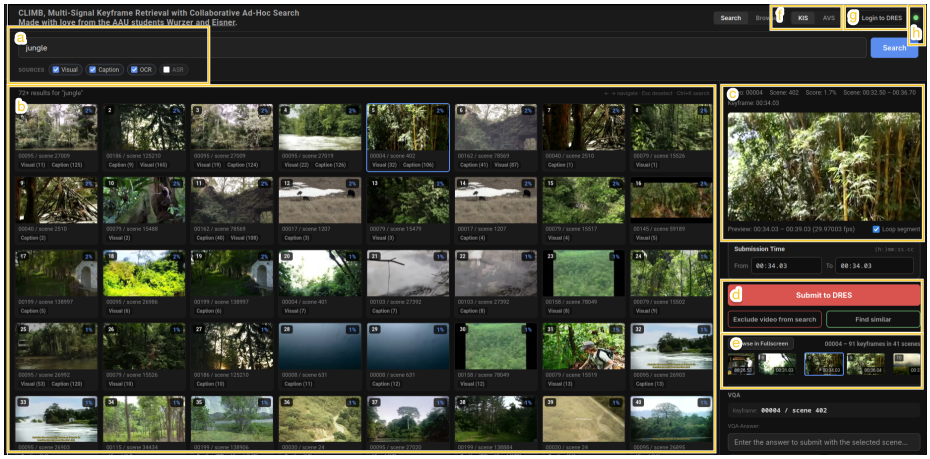


Fig. 2. The CLIMB interface: (a) query bar and source filters, (b) dataset selection, (c) scene result grid, (d) detail panel and video player, (e) submission and search-refinement controls, (f) keyframe filmstrip, (g) AVS session tab, (h) DRES connection controls, (i) Team connection controls, and (j) system health status.

6 Team Collaboration

At VBS, every operator runs a complete local CLIMB instance, to avoid relying on the network for media delivery. Nonetheless, allowing operators to work together is highly beneficial, as teammates can share promising leads to quickly cover a long video or a complex scene, which speeds up the search while reducing wrong or duplicate submissions. Due to the local nature of each instance, coordination requires a shared state outside these instances: a lightweight collaboration service, deployed once. Operators join a team by name and password, with each backend keeping a local copy of its team's state (the current AVS session and all hints), refreshed every four seconds with a few kilobytes of JSON.

Hints Regardless of task type, operators can send a teammate a keyframe or an entire video, along with an optional note. The hint appears as a notification, which when clicked opens a browsing view of all keyframes with the sent keyframe selected. The message, no matter if selected or dismissed, is kept in an inbox, so that operators can come back to it later.

Shared AVS Sessions Since AVS scoring rewards distinct correct shots, independently working team members tend to identify the same obvious results first, resulting in a large fraction of duplicate submissions. In CLIMB, a team therefore shares one AVS session, where starting a new one, for the next task, moves every member along. Submitted scenes are hidden from the results of all team members. Other scenes in these videos get marked as covered but remain visible to promote a diverse submission set, while allowing for the submission of multiple distinct scenes from the same video.

Resilience Importantly, searches and submissions must not depend on the speed or availability of the collaboration service. To ensure this, searches read only the local copy which is refreshed every four seconds. Submissions are sent to DRES first, before recording them with the service, ensuring a non-blocking behavior. Lastly, an unreachable service is not treated as an ended session. The last known local state is kept and flagged as offline, so that an unstable connection cannot clear the team's exclusions. Only the exchange of new submissions and hints pauses until the service is reachable again.

7 Preliminary Results

While implemented for the full V3C collection, our reported metrics were measured on a 200-video subset (26 h, 14,345 scenes, 35,558 keyframes).

Compared with exhaustive exact search (73 ms), oversampling 1,000 candidates before exact reranking recovers on average 95.6% of the true top 20 in 17 ms, 2,000 candidates recovers 98.3% in 33 ms, and 4,000 99.4% in 63 ms. The remaining discrepancy is due to the binary quantization rather than the index: the ANN stage ranks by Hamming distance over 1-bit vectors. CLIMB defaults to 2,000 candidates, while keeping this parameter configurable.

Although a far simpler CLIMB prototype won our university's course-internal retrieval competition, VBS presents a much greater challenge. Our latency measurements are limited to a small subset rather than the full-scale collection, and neither MVK nor LHE resembles the imagery SigLIP2 was trained on. Fusing PeskaVLP with SigLIP2 for LHE retrieval raises the share of queries with a relevant frame on the first result page from 46% to 54%. BioCLIP 2.5 alone, dropped the share of queries with a relevant frame on the first result page from 93% to 64% for MVK and was therefore not used in the final system.

8 Conclusion

In this paper, we present CLIMB, an interactive video retrieval system for VBS 2027. It uses rank-based fusion to combine visual embeddings, on-screen text, generated captions, and transcripts at the keyframe level while supporting temporally chained queries. A collaborative mode hides scenes a teammate already submitted from everyone's results, while search, browsing, and submission remain unaffected when the shared collaboration service is unavailable.

References

1. Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., et al.: Qwen3-VL technical report. arXiv preprint arXiv:2511.21631 (2025)
2. Cormack, G.V., Clarke, C.L.A., Buettcher, S.: Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. p. 758–759. SIGIR '09, Association for Computing Machinery, New York, NY, USA (2009). https://doi.org/10.1145/1571941.1572114, https://doi.org/10.1145/1571941.1572114
3. Gonzalez, T.F.: Clustering to minimize the maximum intercluster distance. Theoretical Computer Science **38**, 293–306 (1985). https://doi.org/https://doi.org/10.1016/0304-3975(85)90224-5, https://www.sciencedirect.com/science/article/pii/0304397585902245
4. Gu, J., Stevens, S., Campolongo, E., Thompson, M., Zhang, N., Wu, J., Kopanev, A., Mai, Z., White, A., Balhoff, J., et al.: BioCLIP 2: Emergent properties from scaling hierarchical contrastive learning. Advances in Neural Information Processing Systems **38**, 102778–102811 (2025)
5. Lokoč, J., Andreadis, S., Bailer, W., Duane, A., Gurrin, C., Ma, Z., Messina, N., Nguyen, T.N., Peška, L., Rossetto, L., et al.: Interactive video retrieval in the age of effective joint embedding deep models: lessons from the 11th VBS. Multimedia Systems pp. 1–24 (2023)
6. Lokoč, J., Bailer, W., Schoeffmann, K., Muenzer, B., Awad, G.: On influential trends in interactive video retrieval: Video Browser Showdown 2015–2017. IEEE Transactions on Multimedia **20**(12), 3361–3376 (2018). https://doi.org/10.1109/TMM.2018.2830110
7. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence **42**(4), 824–836 (2020). https://doi.org/10.1109/TPAMI.2018.2889473
8. Nasirihaghighi, S., Ghamsarian, N., Peschek, L., Munari, M., Husslein, H., Sznitman, R., Schoeffmann, K.: GynSurg: A comprehensive gynecology laparoscopic surgery dataset. In: Proceedings of the 33rd ACM International Conference on Multimedia. p. 13141–13147. MM '25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3746027.3758267, https://doi.org/10.1145/3746027.3758267
9. Radford, A., Kim, J.W., Xu, T., Brockman, G., Mcleavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision. In: Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., Scarlett, J. (eds.) Proceedings of the 40th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 202, pp. 28492–28518. PMLR (23–29 Jul 2023), https://proceedings.mlr.press/v202/radford23a.html
10. Rossetto, L., Gasser, R., Sauter, L., Bernstein, A., Schuldt, H.: A system for interactive multimedia retrieval evaluations. In: Lokoč, J., Skopal, T., Schoeffmann, K., Mezaris, V., Li, X., Vrochidis, S., Patras, I. (eds.) MultiMedia Modeling. pp. 385–390. Springer International Publishing, Cham (2021)
11. Rossetto, L., Schoeffmann, K., Gurrin, C., Lokoc, J., Bailer, W.: Results of the 2025 Video Browser Showdown. ArXiv **abs/2509.12000** (2025), https://doi.org/10.48550/arXiv.2509.12000

12. Rossetto, L., Schuldt, H., Awad, G., Butt, A.A.: V3C – a research video collection. In: Kompatsiaris, I., Huet, B., Mezaris, V., Gurrin, C., Cheng, W.H., Vrochidis, S. (eds.) MultiMedia Modeling. pp. 349–360. Springer International Publishing, Cham (2019)
13. Rublee, E., Rabaud, V., Konolige, K., Bradski, G.: ORB: An efficient alternative to SIFT or SURF. In: 2011 International Conference on Computer Vision. pp. 2564–2571 (2011). https://doi.org/10.1109/ICCV.2011.6126544
14. Sauter, L., Gasser, R., Schuldt, H., Bernstein, A., Rossetto, L.: Performance evaluation in multimedia retrieval. ACM Transactions on Multimedia Computing, Communications and Applications (Oct 2024). https://doi.org/10.1145/3678881, https://doi.org/10.1145/3678881
15. Schoeffmann, K., Del Fabro, M., Szkaliczki, T., Böszörmenyi, L., Keckstein, J.: Keyframe extraction in endoscopic video. Multimedia Tools and Applications **74**, 11187–11206 (2015)
16. Souček, T., Lokoč, J.: Transnet V2: an effective deep network architecture for fast shot transition detection. arXiv preprint arXiv:2008.04838 (2020)
17. Truong, Q.T., Vu, T.A., Ha, T.S., Lokoč, J., Wong, Y.H., Joneja, A., Yeung, S.K.: Marine Video Kit: a new marine video dataset for content-based analysis and retrieval. In: International conference on multimedia modeling. pp. 539–550. Springer (2023)
18. Tschannen, M., Gritsenko, A., Wang, X., Naeem, M.F., Alabdulmohsin, I.M., Parthasarathy, N., Evans, T., Beyer, L., Xia, Y., Mustafa, B., Hénaff, O., Harmsen, J., Steiner, A., Zhai, X.Q.: SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. ArXiv **abs/2502.14786** (2025), https://arxiv.org/abs/2502.14786
19. Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., Wei, F.: Multilingual E5 text embeddings: A technical report. arXiv preprint arXiv:2402.05672 (2024)
20. Yuan, K., Srivastav, V.K., Navab, N., Padoy, N.: Procedure-aware surgical video-language pretraining with hierarchical knowledge augmentation. ArXiv **abs/2410.00263** (2024), https://api.semanticscholar.org/CorpusID:273022518
21. Zhang, Y.B., Wang, X., Lin, M.H., Zhang, Y., Deng, P., Sun, T., Gao, T., Zhang, Z.L., Liu, J.X., Zhou, C., Liu, H., Liang, S., Cui, C., Liu, Y., Yu, D.H., Ma, Y.J.: PP-OCRv6: From 1.5m to 34.5m parameters, surpassing billion-scale vlms on ocr tasks. ArXiv **abs/2606.13108** (2026)